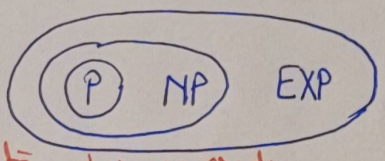
PTIME (P) = mulțimea de probleme ce pot fi decise în timp polinomial

NPTIME (NP) = ——— nedeterministic polinomial

EXPTIME (EXP) = ——— exponențial

$P \subsetneq NP \subsetneq EXP$




$\neq$ → nu e încă demonstrată, dar se crede

Reducere polinomială

$A \leq_p B \iff \exists$ o transformare POLINOMIALĂ $T$ a.î.
$\forall i : A(i) = 1 \iff B(T(i)) = 1$

- $f \leq_p g$, $g \notin P$ $\not\Rightarrow f \notin P$

  ↳ A este reductibilă polinomial la B
  B este cel puțin la fel de grea ca A

- $f \leq_p g$, $g \in P$ $\not\Rightarrow f \in P$

NP-Hard (Dificilă) = { probleme $f$ | $\forall g \in NP$, $g \leq_p f$ }

NP-Complet (Completitudine) = { $f$ | $f \in NPH$ și $f \in NP$ }

- Referitor la faptul că $P \neq NP$ nu a fost încă demonstrată:

  Dacă $f \in NPC$

  I: $P \subsetneq NP \implies f \notin P$

  II: $P = NP \implies f \in P$

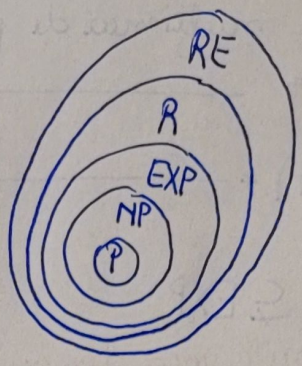
  Până acum, se crede cu tărie că $P \subsetneq NP$ (Nu s-a găsit vreun contraexemplu)

- $SAT \in NPC$
- $f \in NPC$, $g \in NPC$ $\not\Rightarrow g \leq_p f$ și $f \leq_p g$, adică $f \equiv_p g$
- $f \in NPC$, $SAT \in NPC$ $\not\Rightarrow f \leq_p SAT$, $SAT \leq_p f$

  (Pentru că ∄ algoritmi eficienți ca să rezolve SAT, se preferă reducerea problemelor NPC la SAT)

De știut pentru examen:

$f \in R$
$f \in RE$ } → dem. cu algoritm
$f \in EXP$
$f \in NP$
$f \in P$

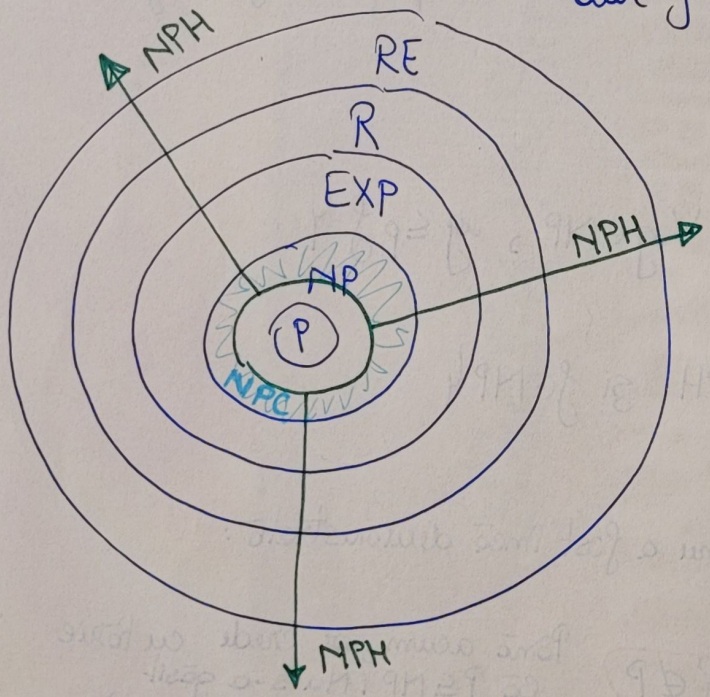



$f \notin R$ → dem. prin reducere $g \leq_T f$ unde $g \notin R$

$f \notin P$ → dem. prin arătare că $f \in NPH$ sau $f \in NPC$

↳ $f \in NPH$ → $SAT \leq_p f$ (invers față de reducerile de până acum)
$\in NPH$

ex Găsiți un exemplu de problemă $f \in NPH$ dar $f \notin NP$



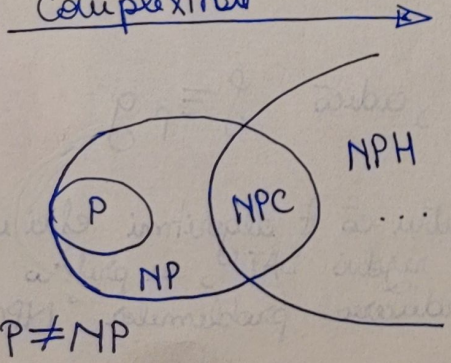

Nu putem demonstra direct că o problemă $\notin P$. Putem doar arăta că $\in NPH$. Dacă $P \neq NP$, înseamnă că problema $\notin P$.

$f_h \in RE \setminus R$
↳ Halting Problem

→ $SAT \in NPH$ se poate reduce polinomial la $f_h$

$SAT \leq_p f_h$
$\Rightarrow f_h \in NPH$
dar nu e în NP

Mai clar ca diagramă:




Obs dacă $P = NP$
$\Rightarrow P = NP = NPC$

aka SAT-ul se rezolvă în timp polinomial

Dar se crede că nu e adevărat!

ex 1

**Partitioning** ($\in$ NPC)

$A = [a_1, a_2 \dots a_n]$

$\exists S \subseteq [1,2\dots,n]$ a.î. $\sum_{i \in S} a_i = \sum_{i \notin S} a_i$ ?

ex: $[1,3,2,6]$

$S = [1,2,3]$

$1+3+2 = 6$

ex $[1\ 5\ 8\ 4]$

$S = [1,3]$

$1+8 = 5+4$

**Subset Sum** ($\in$ NPC)

$B = [b_1, b_2 \dots b_m]$, $K \in \mathbb{N}$

$\exists S \subseteq [1,2\dots,m]$ a.î. $\sum_{i \in S} b_i = K$ ?

a) Partitioning $\leq_p$ Subset Sum

$A \xrightarrow{T} (B,K)$ unde T se calculează în timp polinomial

Dem. că: $P(A) = 1 \iff SS(B,K) = 1$

$T(A)$: $B = A$ ← polinomială (copierea vectorului A în B: $O(n)$) } $O(n)$

$K = \text{sum}(A)/2$ ← $O(1)$

"$\Rightarrow$" $P(A) = 1 \Rightarrow \exists S$ a.î. $\sum_{i \in S} a_i = \sum_{i \notin S} a_i = \frac{\text{sum}(A)}{2}$

$\Rightarrow \sum_{i \in S} a_i = \frac{\text{sum}(A)}{2}$

T: Elementele din B sunt exact cele din A } $\Rightarrow \sum_{i \in S} b_i = \frac{\text{sum}(B)}{2} = K$

$\Rightarrow SS(B,K) = 1$

"$\Leftarrow$" $SS(B,K) = 1 \Rightarrow \exists S$ a.î. $\sum_{i \in S} b_i = K$

$\Rightarrow \sum_{i \notin S} b_i = \text{sum}(B) - K \overset{(T)}{=} \text{sum}(A) - \frac{\text{sum}(A)}{2} = \frac{\text{sum}(A)}{2} = \sum_{i \notin S} a_i$

$\Rightarrow \sum_{i \in S} a_i = \sum_{i \notin S} a_i = \frac{\text{sum}(A)}{2} \Rightarrow P(A) = 1$

b) Subset Sum $\leq_p$ Partitioning

$(B,K) \xrightarrow{T} A$

Dem. că $SS(B,K) = 1 \iff P(A) = 1$

$B = [1,3,5,2,7]$

$K = 4$

$\sum_{i \in S} b_i = 4 = K$

$\Rightarrow \sum_{i \notin S} b_i = \text{sum}(B) - K = 18 - 4 = 14$

$\sum_{i \in S} b_i + ? = \sum_{i \notin S} b_i \Rightarrow ? = sum(B) - 2K$

$\sum_{i \in S} b_i = K$

$\sum_{i \notin S} b_i = sum(B) - K$

I: $2K \leq sum(B) \Rightarrow$ adăugăm în partiție elementul $sum(B) - 2K$ și în vector

II: $2K > sum(B) \Rightarrow$ adăugăm în vector elementul $2K - sum(B)$

$\Rightarrow$ În exemplul anterior vom adăuga elementul 10 la vector

T: $A = B \cup \{|-2K + sum(B)|\}$ ← $O(n)$ deci polinomială

$\Rightarrow$ SS(B, K) = 1 $\Rightarrow \exists S$ a.î. $\sum_{i \in S} b_i = K$

$\xRightarrow{I}$ $sum(A) = sum(B) - 2K + sum(B)$ ← pt. caz I

$= 2sum(B) - 2K$ ← partiția ce verifică SS pentru B, K

Alegem partia $S' = S \cup [sum(B) - 2K]$

$\Rightarrow \sum_{i \in S'} a_i = \sum_{i \in S} b_i + sum(B) - 2K =$

$= K$

$= sum(B) - K$

$\sum_{i \notin S'} a_i = \sum_{i \notin S} b_i = sum(B) - \sum_{i \in S} b_i =$

$= sum(B) - K$

Se repetă raționamentul și pentru caz II.

$\Leftarrow$

$P(A) = 1 \Rightarrow \exists S$ a.î. $\sum_{i \in S} a_i = \sum_{i \notin S} a_i$

Pentru caz I:

Fie $S' = \{i \mid i \notin S$ și $i \leq n\}$

Alegem partiția $S''$ ca fiind partiția dintre S și S' care conține elementul $X = sum(B) - 2K$

$\Rightarrow \sum_{S'', i \neq X} a_i = \frac{sum(A)}{2} - X = sum(B) - K - X =$

$= sum(B) - K - (sum(B) - 2K) =$

$= K$

Se repetă raționamentul și pentru caz II.

La curs KVC $\leq_p$ K-Clique

KVC $\in$ NP-H $\longrightarrow$ SAT $\leq_p$ 3SAT $\leq_p$ K-Clique $\leq_p$ KVC

NP (am văzut la cabul anterior)

$\Rightarrow$ NP-C

Set Cover $\in$ NP (algoritm nedeterminist)

NPH $\longrightarrow$ KVC $\leq_p$ Set Cover

$\Rightarrow$ NPC

HamiltonianPath $\in$ NP (alg. nedet.)

NPH $\longrightarrow$ 3SAT $\leq_p$ HamiltonianPath

$\Rightarrow$ NPC

SAT $\in$ NP (alg. nedet.)

NPH $\longrightarrow$ SAT is NPH (Cook Levine theorem)

$\Rightarrow$ NPC

---

ex 2

## SAT $\leq_p$ 3-SAT

3SAT = SAT în care fiecare clauză are exact 3 literali

$\varphi$ în CNF $\longrightarrow$ $\varphi'$ în 3CNF

T: Idee: spargem clauzele cu mai mult de 3 literali în clauze cu fix 3.

$I' = I$ dacă $I \models$ clauza inițială

1) Clauze cu 1 literal: inventăm 2 variabile ce nu influențează satisfiabilitatea

$\overline{x_1} \xrightarrow{T} (\overline{x_1} \vee y_1 \vee y_2) \wedge (\overline{x_1} \vee \overline{y_1} \vee \overline{y_2}) \wedge (\overline{x_1} \vee \overline{y_1} \vee y_2) \wedge (\overline{x_1} \vee y_1 \vee \overline{y_2})$

2) Clauze cu 2 literali: inventăm o variabilă ce nu influențează satisfiabilitatea

$x_1 \vee \overline{x_2} \xrightarrow{T} (x_1 \vee \overline{x_2} \vee y) \wedge (x_1 \vee \overline{x_2} \vee \overline{y})$

3) Clauze cu 3 literali: rămân neschimbate

4) Clauze cu 4+ literali

$(x_1 \vee \overline{x_2} \vee x_3 \vee \overline{x_4} \vee x_5) \xrightarrow{T} (x_1 \vee \overline{x_2} \vee y_1) \wedge (\overline{y_1} \vee x_3 \vee y_2) \wedge$
$\wedge (\overline{y_2} \vee \overline{x_4} \vee x_5)$

dc. $I \models x_1 \vee \overline{x_2}$
$\Rightarrow I' = I \quad (y_1 = F, y_2 = F)$

dc $I \models x_3$
$\Rightarrow I' = I \cup \{y_1\}$

dc. $I \models \overline{x_4} \vee \overline{x_5}$
$\Rightarrow I' = I \cup \{y_1, y_2\}$

$\Rightarrow$ Caz general: $C = L_1 \vee L_2 \vee \dots \vee L_n$ cu $L = x$ sau $L = \overline{x}$

$\rightarrow$ construim $n-3$ variabile aditionale

$\rightarrow$ construim formula

$(L_1 \vee L_2 \vee y_1) \wedge (\overline{y_1} \vee L_3 \vee y_2) \wedge$
$\wedge \dots \wedge (\overline{y_{i-2}} \vee L_i \vee y_{i-1}) \wedge \dots \wedge$
$\wedge (\overline{y_{n-3}} \vee L_{n-1} \vee L_n)$

$SAT(f) = 1 \iff 3SAT(f') = 1$

$T$ ales este polinomial : $O(n^2)$ clauze generate

$\Rightarrow$ $SAT(f) = 1 \Rightarrow f$ satisfiabilă $\Rightarrow \exists I \models C$

Clauze cu 1/2/3 literali : $I' = I$

Clauze cu 4 literali:

$I \models L_1 \vee L_2 \Rightarrow I \models T(f) = f'$

$I \models L_i$ cu $3 \le i \le n-2 \Rightarrow I' \models f'$
$I' = \{y_1, \dots, y_{i-2}\} \cup I$

$I \models L_{n-1} \vee L_n \Rightarrow I' = I \cup \{y_1, \dots y_{n-3}\}$

$\Rightarrow f'$ satisfiabilă și în 3CNF
$\Rightarrow 3SAT(f') = 1$

$\Leftarrow$ 3SAT($\varphi'$) = 1 $\Rightarrow$ $\varphi'$ satisfiabilă $\xrightarrow{I}$ $I = I' / \{$ stările Y4

$\Longrightarrow$ (raționamentul exact ca fel ca mai devreme, dar invers) $I \models \varphi$

$\Rightarrow$ SAT($\varphi$) = 1.

---

ex3 K Clique : Există un subgraf complet de 4 noduri în graful dat?

↳ $G = (V, E)$ ; $K \in \mathbb{N}$

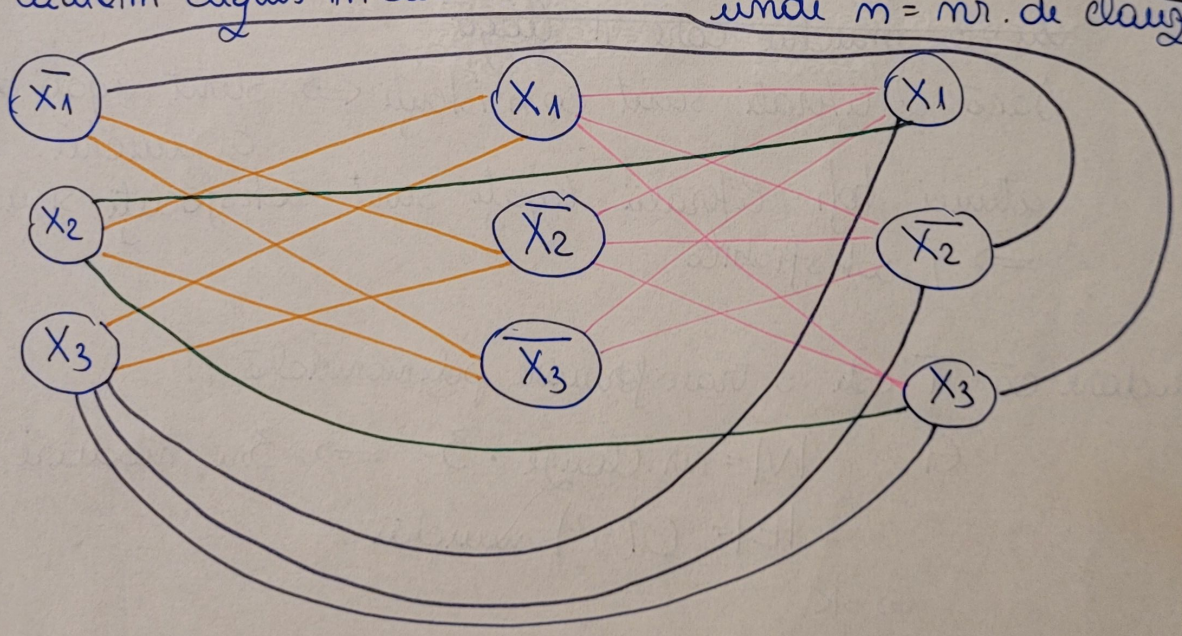
3SAT $\leq_p$ K Clique

$\varphi \xrightarrow{T} G = (V, E)$ ; $K$

$(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3)$

T: - între nodurile aceleiași clauze nu se pun muchii

- Se pun muchii între nodurile a 2 clauze diferite care sunt consistente unul cu altul (toate exceptând $x - \overline{x}$)
- căutăm cliques în care toate clauzele sunt posibile simultan $\rightarrow$ $K = n$ unde $n$ = nr. de clauze




$V = \{ \alpha_{ij}$ pentru fiecare $L_{ij}$ din fiecare clauză $i \}$

$(\alpha_{ij}, \alpha_{uv}) \in E$ dacă $i \neq u$ (din clauze diferite)

și $L_{ij} \neq \overline{L_{uv}}$ (corespund unor literali consistenți)

$K$ = nr. de clauze.

3SAT(f) = 1 $\Longleftrightarrow$ KClique ( G = (V, E), K) = 1

$\Longrightarrow$ Dacă $C_1 \wedge C_2 \dots \wedge C_n$ satisfiabilă

$\Longrightarrow$ G are o n-clică

Pp. $I \models f \Rightarrow I \models C_i$ cu $i = \overline{1,2 \dots n}$

$\Longrightarrow$ în fiecare clauză $C_i$, cel puțin un literal este consistent cu ceilalți

$\Longrightarrow$ nodurile ce corespund acestor literali consistenți au muchii între ele $\Longrightarrow$ formează o n-clică

$\Longleftarrow$ Pp. că $\exists$ o $K$-clică ($K = n$) în G. Ea conține noduri inter-clauze care nu pot fi decât n ce număr.

Dacă 2 literali sunt consistenți $\Longleftrightarrow$ sunt legați de o muchie,

atunci toți literalii legați sunt satisfăcuți simultan

$\Longrightarrow$ f satisfiabilă

Validare că T este o transformare polinomială

G: $|V| = \text{nr. clauze} \cdot 3 \Longrightarrow 3n$ noduri

$|E| = O(n^2)$ muchii

$\Rightarrow$ ok

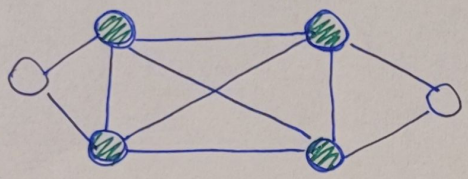
ex 4

# $3SAT \leq_p KVC$

Știm deja $3SAT \leq_p K\,Clique$

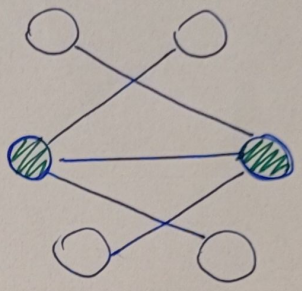
Încercăm: $KClique \leq_p KVC$

Idee: clique = subgraf complet



4-clique

$\xrightarrow{T}$



2-cover

$\Updownarrow$

$|V|-K$

$$T\begin{cases} G_1 = (V_1, E_1), K_1 \xrightarrow{T} G_2 = (V_2, E_2), K_2 \\ E_2 = \{(u,v) \in V \times V \mid (u,v) \notin E_1\} = \overline{E_1} \\ V_2 = V_1 \\ K_2 = |V_1| - K_1 \end{cases}$$

[ $G_1$ are o K-clique $\Longleftrightarrow$ $G_2 = \overline{G_1}$ are o $(|V|-K)$ acoperire ]

T se poate determina în timp polinomial $O(|V|^2)$